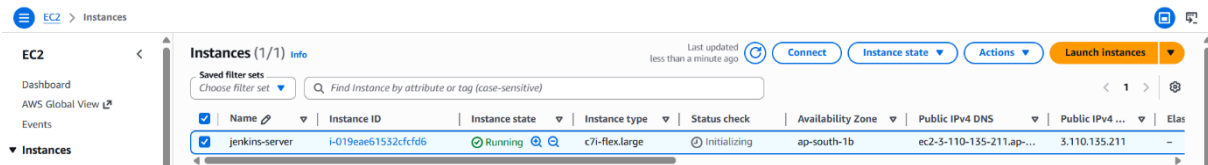


HANDS-ON - EXECUTING A JAVA PROJECT IN JENKINS

Screenshot: Create Ubuntu EC2 Instance

- Launch a new Ubuntu EC2 instance in AWS.
- Configure the required instance settings and security group.



Screenshot: Install Java on Ubuntu

- Execute `sudo su`.
- Install Java using `apt install openjdk-17-jdk -y`.

```
root@ip-172-31-2-102:/home/ubuntu# sudo apt update
sudo apt install fontconfig openjdk-21-jre
java -version
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
39 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
  fontconfig  openjdk-21-jre
Installing dependencies:
  adwaita-icon-theme  gsettings-desktop-schemas  libavahi-common3  libgl1  libibus-1.0-5
  alsa-topology-conf  gtk-update-icon-cache  libcairo-gobject2  libgl1-mesa-dri  libice6
```

Screenshot: Install and Connect Jenkins

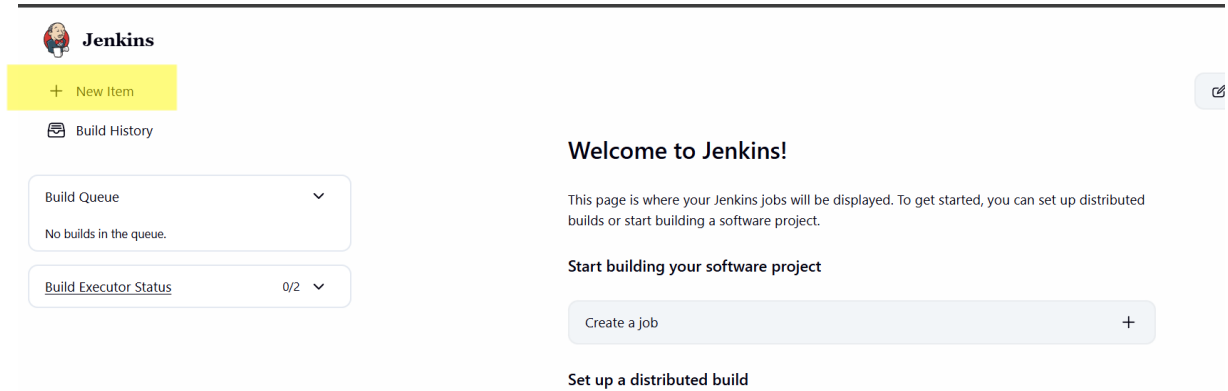
- Install Jenkins using `apt install jenkins -y`.
- Start the Jenkins service.
- Open the Jenkins web interface using `<Public-IP>:8080`.
- Jenkins connected successfully.

```
root@ip-172-31-2-102:/home/ubuntu#
root@ip-172-31-2-102:/home/ubuntu# sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install jenkins
--2026-06-02 03:50:07-- https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
Resolving pkg.jenkins.io (pkg.jenkins.io)... 151.101.210.133, 2a04:4e42:31::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)[151.101.210.133]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1680 (1.6K) [application/octet-stream]
Saving to: '/etc/apt/keyrings/jenkins-keyring.asc'

/etc/apt/keyrings/jenkins-keyring.asc 100%[=====
```

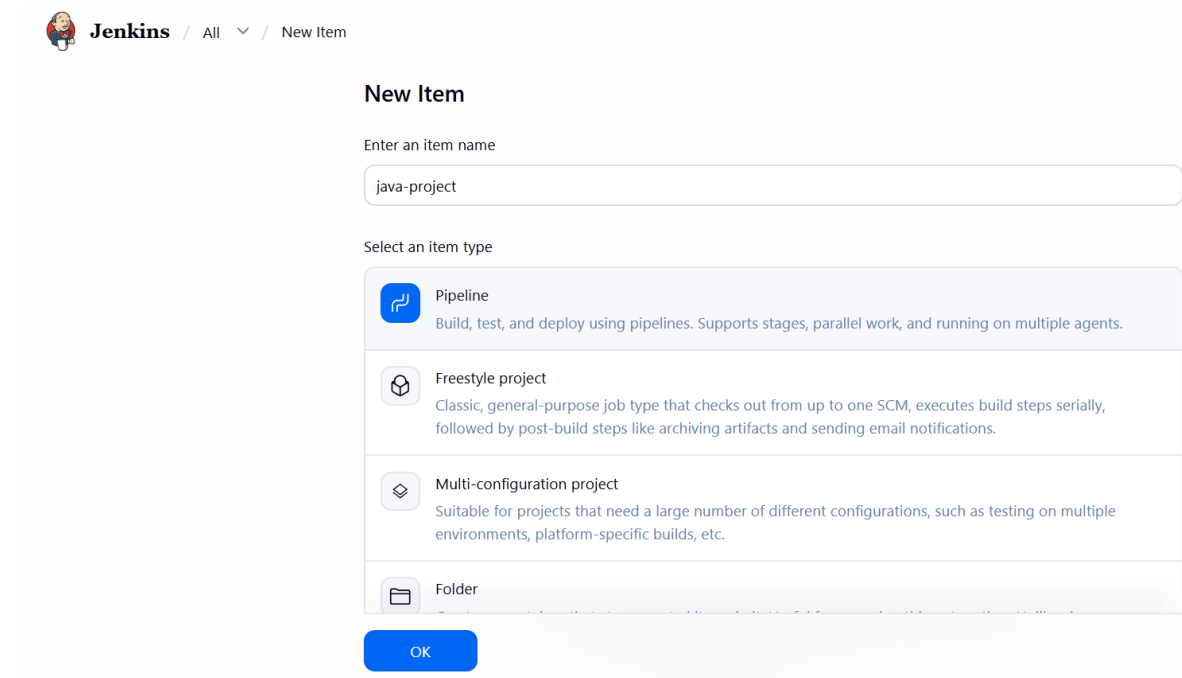
Screenshot: Create a New Jenkins Item

- Click **New Item**.
- New Item creation page opened successfully.



Screenshot: Create Java Pipeline Project

- Enter the project name as **Java Project**.
- Select **Pipeline**.
- Click **OK**.



Screenshot: Configure Pipeline Definition

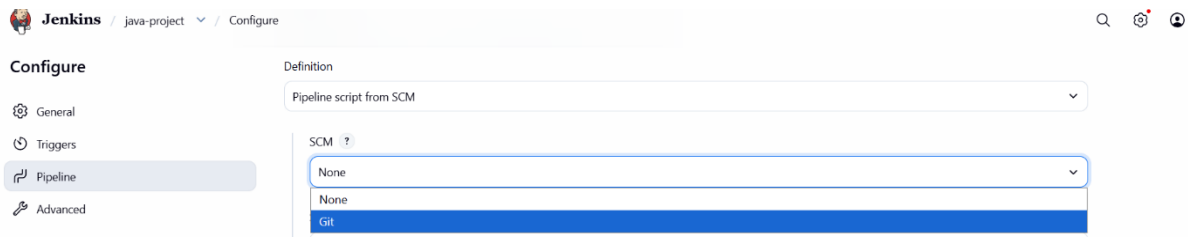
- Scroll down to the **Pipeline** section.

- In **Definition**, select **Pipeline script from SCM**.



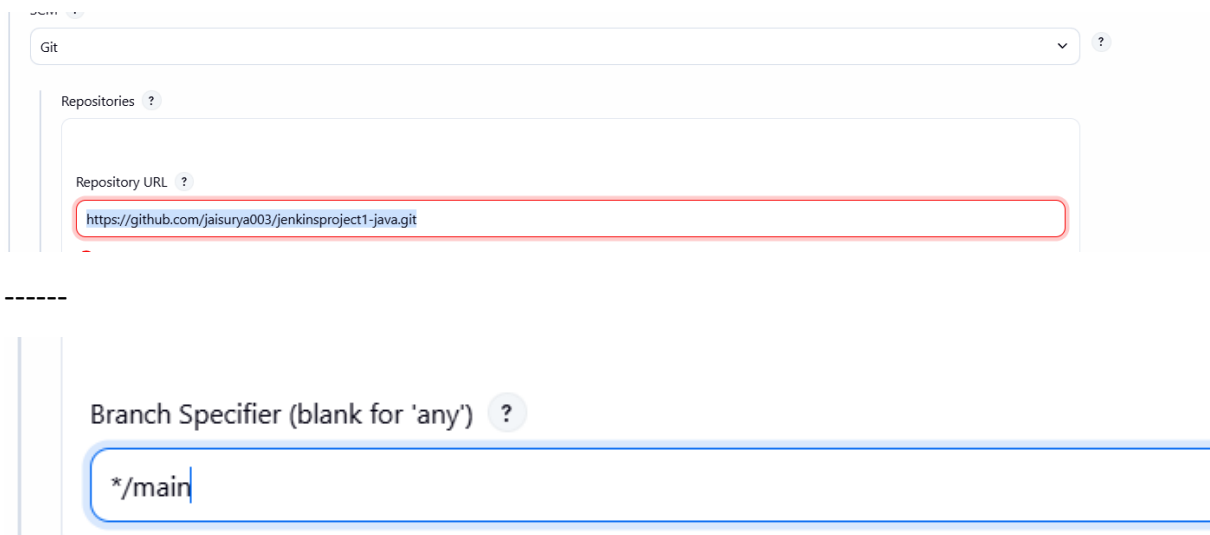
Screenshot: Select Git as SCM

- In the **SCM** field, select **Git**.



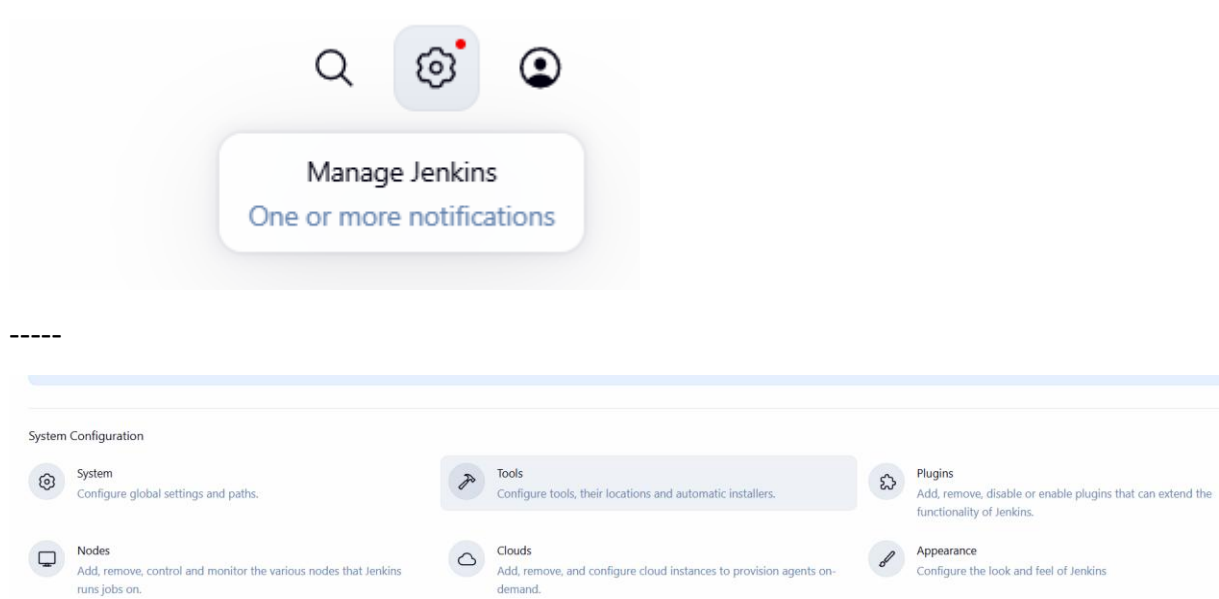
Screenshot: Configure Git Repository and Save

- Paste the project repository URL in the **Repository URL** field.
- Set the **Branch Specifier** as main.
- Click **Save**.



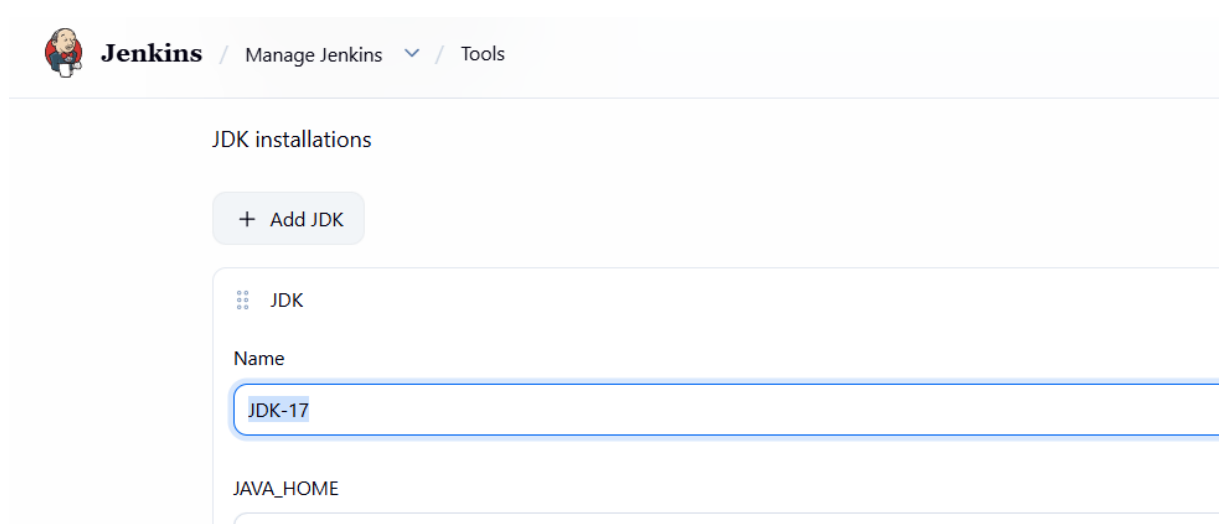
Screenshot: Open Jenkins Tools Configuration

- Click the **Settings** icon in the top-right corner.
- Select **Tools**.



Screenshot: Configure JDK and Maven Tools

- In the **JDK** section, click **Add JDK**.
- Enter the JDK name as **JDK-17**.
- In the **Maven** section, click **Add Maven**.
- Enter the Maven name as **Maven-3.9**.
- Click **Apply** and **Save**.
- JDK and Maven tools configured successfully.



Maven installations

+ Add Maven

Maven

Name

Maven-3.9

☒ Install automatically ?

Install from Apache

Version

3.9.16

Save

Apply

Screenshot: Open Java Project Pipeline

- Click the **Java Project** pipeline.
- Java pipeline project opened successfully.



Jenkins

+ New Item

Build History

Build Queue

No builds in the queue.

Build Executor Status

0/2

All +

S	W	Name ↓	Last Success
...	☀	java-project	N/A

Screenshot: Build Java Project

- Click **Build Now**.
- Jenkins starts the pipeline execution.



Jenkins / java-project



Status



Changes



Build Now



Configure



Delete Pipeline



Stages

java-project

Permalinks

Screenshot: View Build Console Output

- Click the completed **Build**.
- Select **Console Output**.
- Pipeline execution logs are displayed successfully.



Jenkins / java-project



Status



java-project



Changes



Build Now



Configure



Delete Pipeline



Stages



Rename



Pipeline Syntax

</> Changes

Console Output

Edit Build Information

Delete build '#1'

Timings

Git Build Data

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

5 sec ago

(#1), 35 sec ago

build (#1), 35 sec ago

build (#1), 35 sec ago

Builds >



Filter

Today



#1 11:25 AM

Screenshot: Successful Build Console Output

- Open the completed **Build**.
- Click **Console Output**.
- Build execution logs are displayed successfully.
- Output shows **BUILD SUCCESS**.

```
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.637 s
[INFO] Finished at: 2026-06-04T11:25:34Z
[INFO] -----
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Build Successful!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Screenshot: Verify Jenkins Workspace

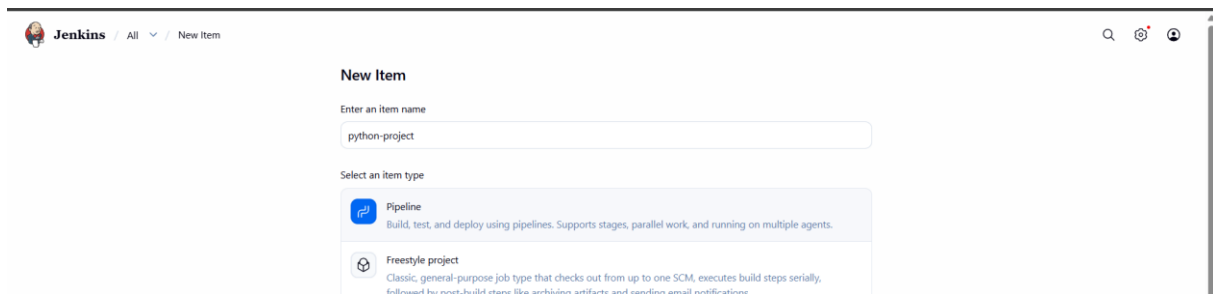
- Navigate to the Jenkins workspace using `cd /var/lib/jenkins/workspace/JavaProject`.
- List the project files using `ls`.
- Jenkinsfile is displayed successfully in the workspace.

```
root@ip-172-31-14-76:/var/lib/jenkins#
root@ip-172-31-14-76:/var/lib/jenkins# cd /var/lib/jenkins/workspace/java-project
root@ip-172-31-14-76:/var/lib/jenkins/workspace/java-project# ls
Jenkinsfile  README.md  pom.xml  src  target
root@ip-172-31-14-76:/var/lib/jenkins/workspace/java-project#
```

HANDS-ON -- EXECUTING A PYTHON PROJECT IN JENKINS AND PUSHING THE DOCKER IMAGE TO DOCKER HUB

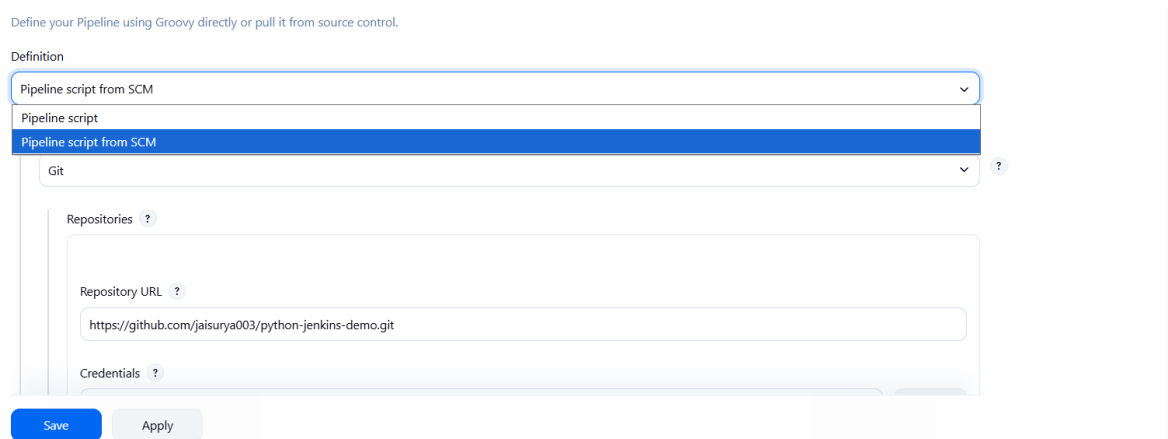
Screenshot: Create Python Pipeline Project

- Click New Item.
- Enter the project name as Python Project.
- Select Pipeline.
- Click OK.



Screenshot: Configure Python Pipeline from SCM

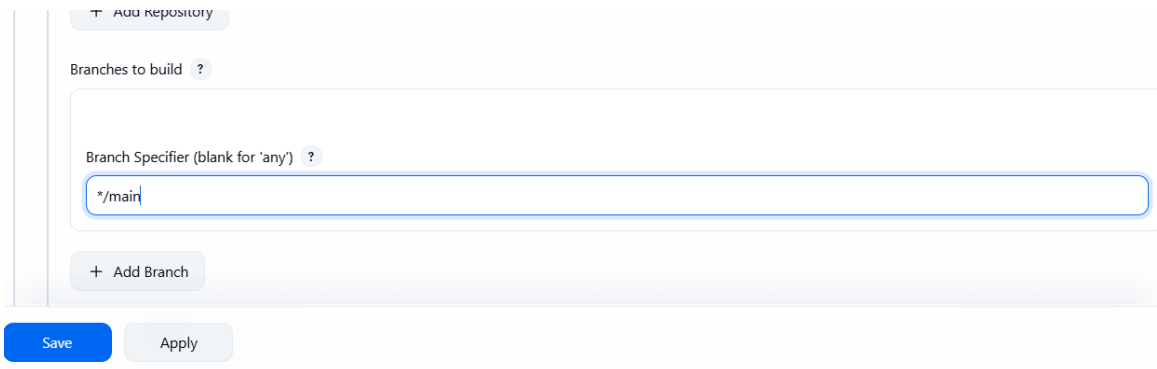
- In the **Definition** field, select **Pipeline script from SCM**.
- In the **SCM** field, select **Git**.
- Paste the Python project repository URL in the **Repository URL** field.



Screenshot: Configure Branch and Save

- Set the **Branch Specifier** as main.

- Click **Apply**.
- Click **Save**.



The screenshot shows the 'Add repository' section of the Jenkins configuration page. It includes a 'Branches to build' section with a 'Branch Specifier (blank for \'any\')' input field containing the text '*/main'. Below this is an 'Add Branch' button. At the bottom of the form are 'Save' and 'Apply' buttons.

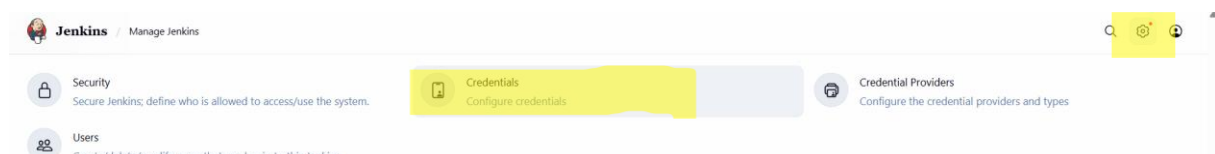
Screenshot: Install Docker and Python

- Run the following commands to install Docker and Python on the server.

```
20 sudo apt update
21 sudo apt install docker.io -y
22 sudo systemctl start docker
23 sudo systemctl enable docker
24 sudo usermod -aG docker jenkins
25 sudo systemctl status docker
26 sudo systemctl restart jenkins
27 sudo apt update
28 sudo apt install python3-venv -y
29 sudo systemctl restart jenkins
30 python3 -m venv testenv
31 history
root@ip-172-31-14-76:/home/ubuntu#
```

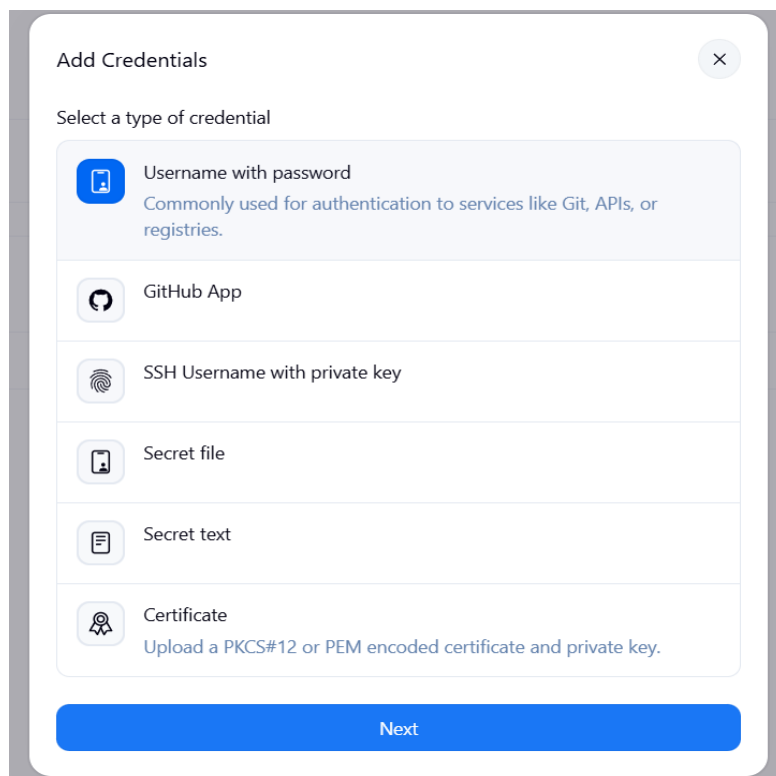
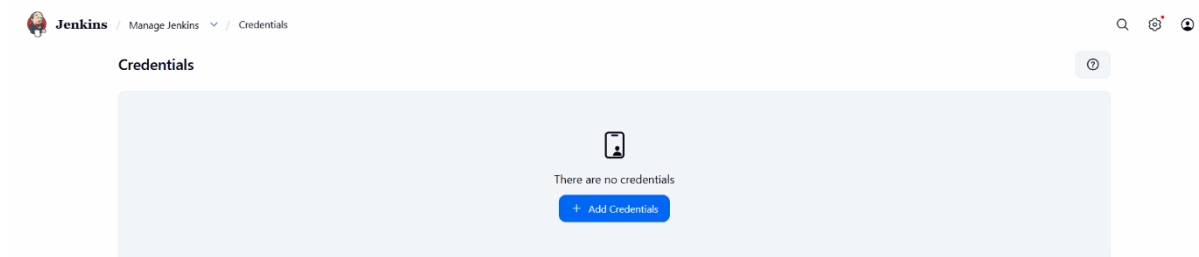
Screenshot: Open Jenkins Credentials

- Go to the Jenkins dashboard.
- Click the **Settings** icon.
- Scroll down to the **Credentials** section



Screenshot: Add Jenkins Credentials

- Click + Add Credentials.
- Select Username with password.



Screenshot: Create Docker Hub Credentials

- Enter the Docker Hub Username.
- Enter the Docker Hub Password.
- Open the Jenkinsfile and copy the value of credentialsId (e.g., docker_cred).
- Paste it into the ID field.
- Click Create.

<

Add Username with password

×

Scope ?
Global (Jenkins, nodes, items, all child items, etc) ▼

Username
jaisurya07

☐ Treat username as secret ?

Password
.....

ID ?
docker-creds

Description ?

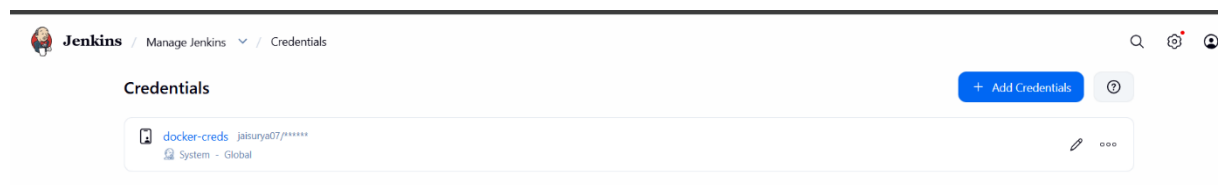
Create

(credential id)

```
stage('Build & Push Docker Image') {  
  steps {  
    withCredentials([usernamePassword(  
      credentialsId: 'docker-creds',  
      usernameVariable: 'USER',  
      passwordVariable: 'PASS'  
    )]) {  
      // ...  
    }  
  }  
}
```

Screenshot: Docker Hub Credentials Created

- Docker Hub credentials created successfully in Jenkin



Screenshot: Open Python Project Pipeline

- Click the **Python Project** pipeline.
- Python pipeline project opened successfully.

All						
S	W	Name ↓	Last Success	Last Failure	Last Duration	
✓	☀	java-project	39 min #1	N/A	31 sec	
⋮	☀	python-project ▾	N/A	N/A	N/A	

Screenshot: Build Python Project

- Click **Build Now**.
- Python pipeline execution started successfully.

 **Jenkins** / python-project

 Status

 Changes

 Build Now

 Configure

 Delete Pipeline

 Stages

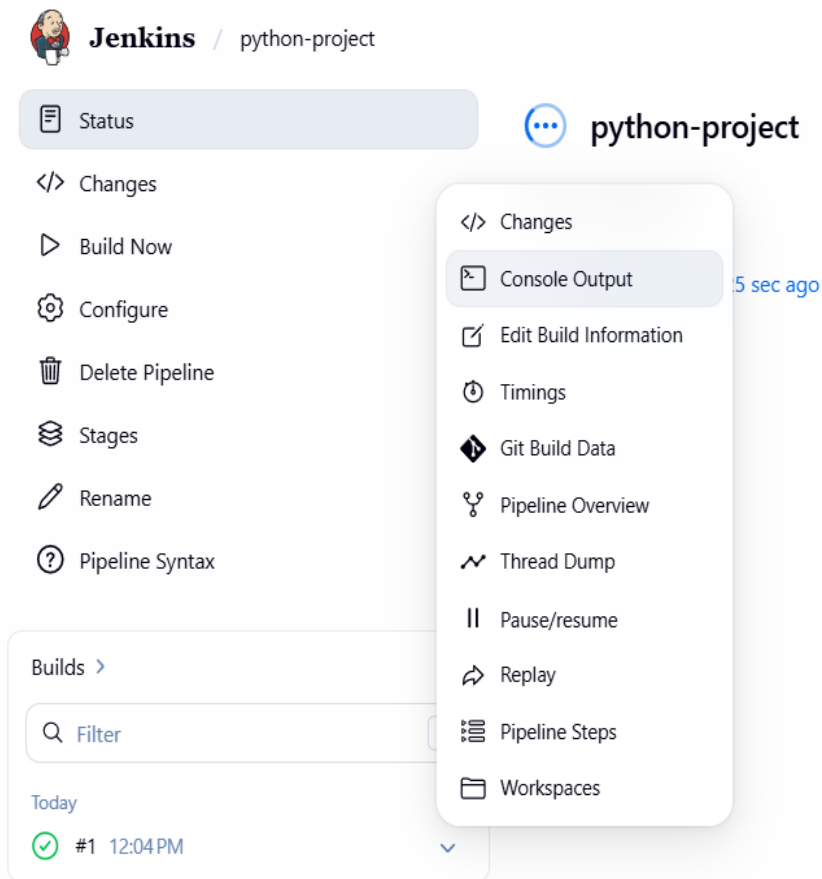
 Rename

 Pipeline Syntax

Screenshot: View Python Build Console Output

- Click the completed **Build**.
- Select **Console Output**.
- Pipeline execution logs are displayed successfully.

- Python project executed successfully and Docker image pushed to Docker Hub.



Screenshot: Successful Pipeline Execution

- Click the completed **Build**.
- Select **Console Output**.
- Build completed successfully.
- Test completed successfully.
- Deployment completed successfully.

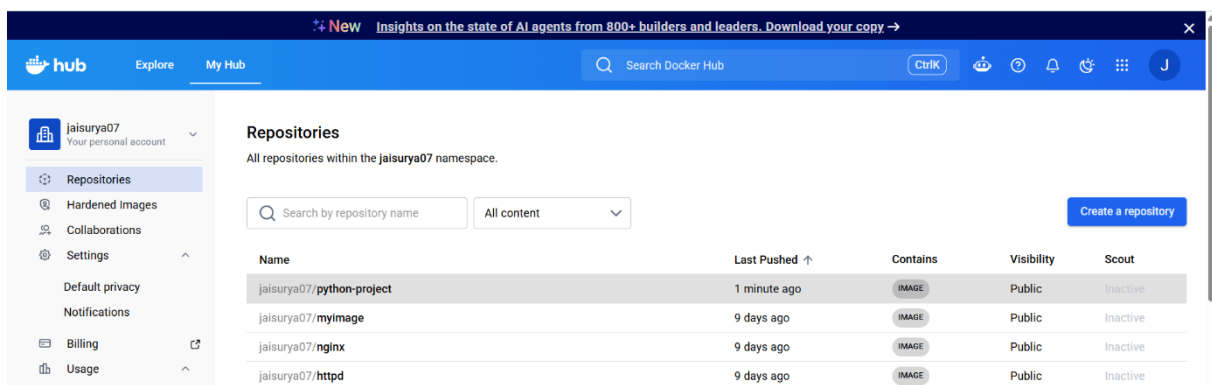
```

[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
🚀 Deploy step (optional)
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
✅ Success: jaisurya07/python-project:1
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

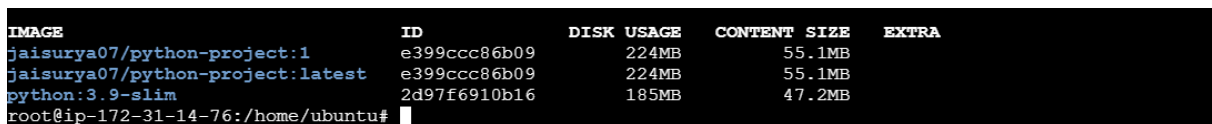
Screenshot: Verify Docker Hub Image

- Open **Docker Hub**.
- Navigate to **Repositories**.
- Python project Docker image is displayed successfully.



Screenshot: Verify Docker Images in Terminal

- Run docker images.
- Python project Docker image is displayed successfully.



CONFIGURING GITHUB WEBHOOK FOR AUTOMATIC JENKINS BUILD TRIGGER

WEBHOOK

A **Webhook** is a mechanism that automatically sends information from one application to another when a specific event occurs. In Jenkins, a webhook is commonly used to trigger a build automatically whenever code is pushed to a Git repository, enabling Continuous Integration (CI).

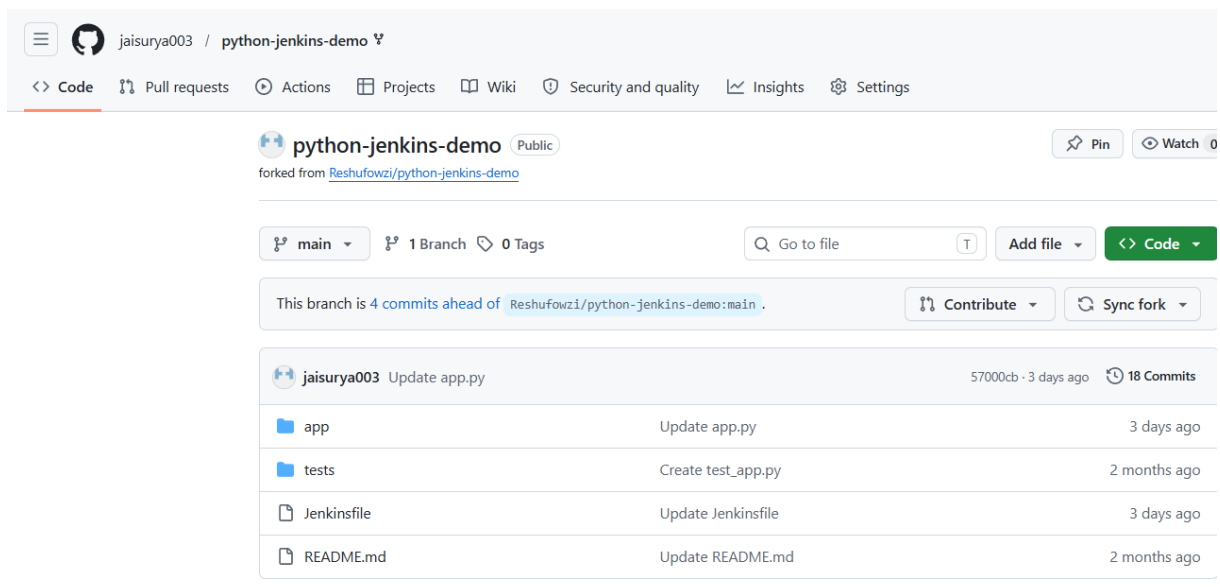
Uses of Webhook

- Automatically triggers a Jenkins build when code is pushed to a Git repository.
- Eliminates manual build execution, enabling faster Continuous Integration (CI).

HANDS-ON

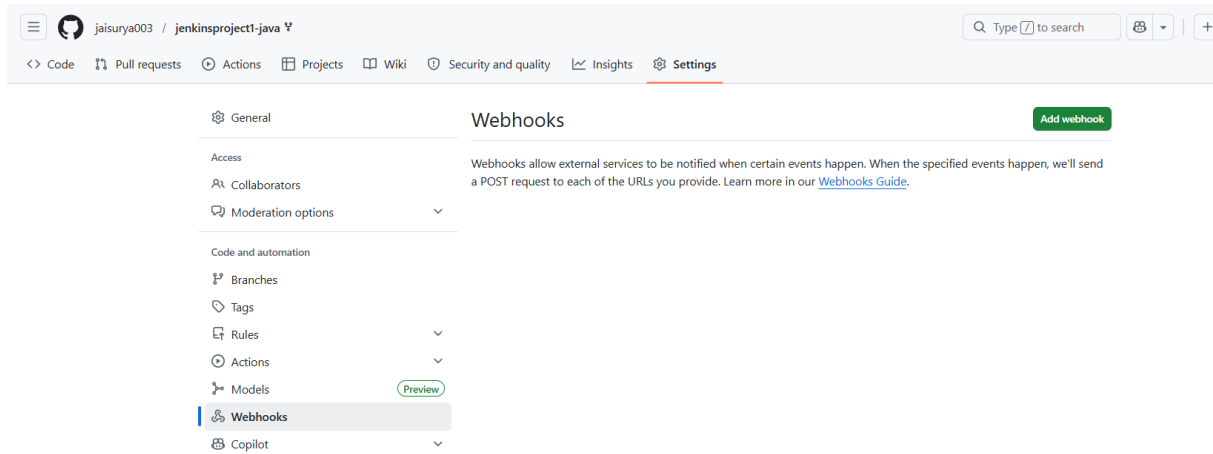
Screenshot: Open GitHub Repository Settings

- Open the **python Project** repository in GitHub.
- Click the **Settings** tab.
- Repository settings page opened successfully.



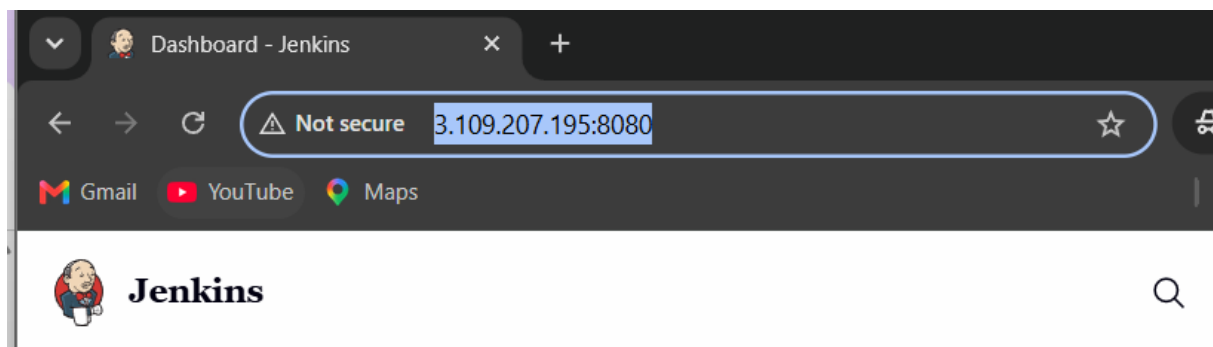
Screenshot: Open Webhooks

- Click **Webhooks** in the repository settings.
- Webhooks page opened successfully.



Screenshot: Copy Jenkins Dashboard URL

- Open the **Jenkins Dashboard**.
- Copy the Jenkins dashboard URL from the browser address bar.



Screenshot: Configure Webhook Payload

- In the **Payload URL** field, paste the Jenkins URL followed by `/github-webhook/`.
- In **Content type**, select **application/json**.
- Webhook payload configured successfully.

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Models

Webhooks

Copilot

Environments

Webhooks / Add webhook

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *

Content type *

Secret

SSL verification
☒ By default, we verify SSL certificates when delivering payloads.

Payload URL *

http://13.201.128.96:8080/github-webhook

Content type *

application/x-www-form-urlencoded

application/json

application/x-www-form-urlencoded

Screenshot: Configure Webhook Events

- Select **Send me everything** in the **Which events would you like to trigger this webhook?** section.
- Click **Add webhook**.

Which events would you like to trigger this webhook?

☐ Just the push event.

☒ **Send me everything.**

☐ Let me select individual events.

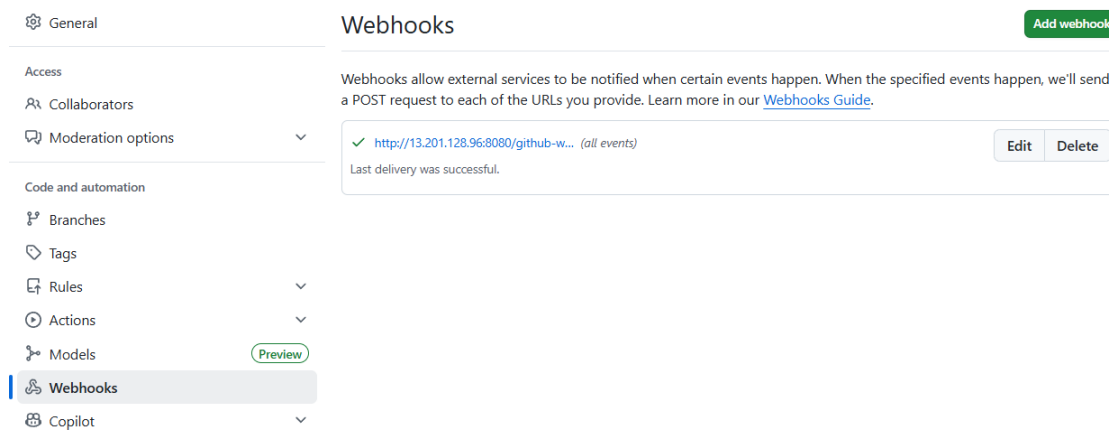
☒ **Active**

We will deliver event details when this hook is triggered.

Add webhook

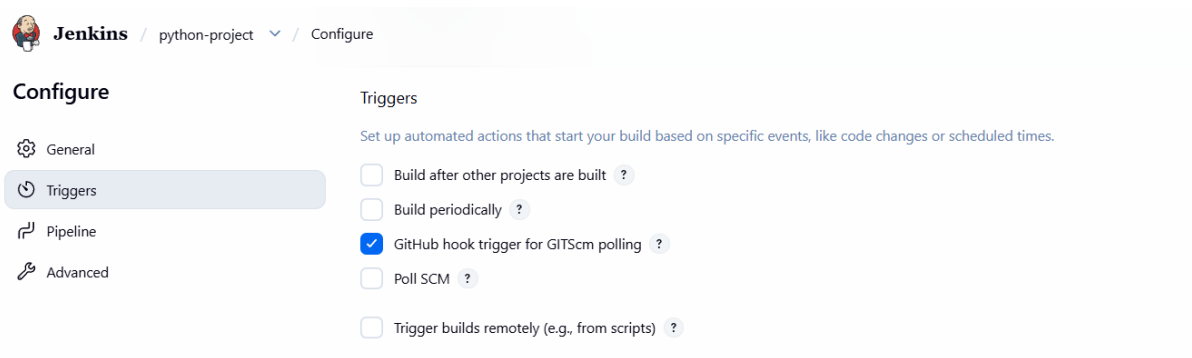
Screenshot: Webhook Created Successfully

- Click **Add webhook**.
- Webhook created successfully and added to the repository.



Screenshot: Configure GitHub Webhook Trigger in Jenkins

- Go to the **Jenkins Dashboard**.
- Click the **Python Project**.
- Click **Configure**.
- Scroll down to **Build Triggers**.
- Select **GitHub hook trigger for GITScm polling**.
- Click **Save**.



Screenshot: Trigger Webhook by Committing Changes

- Open the **Python Project** repository in GitHub.
- Open the app.py file.
- Click **Edit**.

- Make any changes to the file.
- Click **Commit changes**.



Screenshot: Verify Automatic Build Trigger

- Open the **Python Project** in the Jenkins Dashboard.
- Check the **Build History** section.
- A new build is triggered and starts automatically after the GitHub commit.
- Webhook integration verified successfully.

